

Abstract

The **InvenTrack Inventory Management System** is designed to automate and streamline the process of managing inventory data using a secure and database-driven approach. Traditional inventory management systems, such as manual registers or basic spreadsheets, are time-consuming, error-prone, and inefficient when handling large volumes of data. These systems often lead to data inconsistency, duplication, and difficulties in tracking inventory accurately. In contrast, InvenTrack provides a centralized and automated solution that ensures efficient data handling and improved accuracy.

The system is developed using Python and integrates a MySQL database for structured data storage and management. It utilizes Tkinter for building an interactive graphical user interface (GUI), bcrypt for secure password hashing, and python-dotenv for managing environment variables. The application includes a user authentication system that ensures only authorized users can access and manipulate inventory data. All user credentials are securely stored in encrypted form, enhancing system security.

The system maintains a database of inventory records and user details, allowing operations such as data insertion, retrieval, updating, and deletion. During operation, users interact with the system through a GUI, where they can manage inventory efficiently. The setup script automates the installation process, including dependency installation, database creation, and initial configuration, making the system easy to deploy and use.

This automated approach reduces manual errors, improves data accuracy, and enhances operational efficiency. It also minimizes the time required for inventory tracking and management. The system is designed with a modular architecture, allowing easy scalability and future enhancements such as cloud integration, advanced analytics, and multi-user role management.

CHAPTER -1

INTRODUCTION

1.1 About the Project

The **InvenTrack Inventory Management System** is an intelligent, Python-based desktop application designed to streamline the process of managing inventory through database-driven automation and secure user authentication. In traditional environments, inventory is often maintained using manual registers, spreadsheets, or basic digital tools, which are prone to errors, duplication, data inconsistency, and inefficiency. These conventional techniques require continuous human effort and are not suitable for handling large volumes of data accurately.

As organizations move toward digital transformation, there is a growing need for a reliable, secure, and efficient system that can manage inventory in real time. The proposed system addresses these challenges by providing a centralized platform for storing, managing, and accessing inventory data using a structured database approach.

The concept behind this project is simple yet powerful: inventory data is stored in a secure database, and authorized users can access and manipulate this data through a user-friendly graphical interface. The system ensures that only authenticated users can log in, using encrypted password mechanisms to maintain security. Once logged in, users can interact with inventory records efficiently, reducing manual effort and improving productivity.

Technical Overview

The project is implemented entirely in Python 3.8+, utilizing a combination of open-source libraries and tools that provide comprehensive functionality:

1. **MySQL** — manages structured storage of inventory and user data.
2. **Tkinter** — creates the graphical user interface (GUI) for smooth human–computer interaction.
3. **bcrypt** — provides secure password hashing and authentication.
4. **python-dotenv** — manages environment variables such as database credentials securely.

5. **mysql.connector** — enables communication between Python and the MySQL database.

The software follows a modular architecture consisting of three layers:

- **Presentation Layer (GUI):**
Designed using Tkinter; responsible for displaying windows, forms, and user interaction components.
- **Logic Layer (Processing Engine):**
Handles authentication, data validation, and interaction between the GUI and database.
- **Data Layer (Database Storage):**
Utilizes MySQL to store user credentials and inventory records in a structured format.

This separation ensures that each layer can be maintained or upgraded independently without affecting the others, promoting scalability and reliability.

Functional Description

- **User Authentication Module:**
Users must log in using their email and password. Passwords are securely stored using bcrypt hashing, ensuring data protection. The system also limits login attempts to enhance security.
- **Inventory Management Module:**
This module allows users to add, update, delete, and view inventory items. Each item is stored in the database with relevant details such as name, quantity, and category.
- **Database Management Module:**
Handles all database operations, including insertion, retrieval, updating, and deletion of records using MySQL queries.
- **Setup Module:**
A dedicated setup script automates installation, database creation, and initialization of the system, making deployment easy and efficient.

System Advantages

- **Automation:** Eliminates manual record-keeping and reduces human effort.
- **Accuracy:** Ensures structured and error-free data storage.
- **Security:** Uses encrypted passwords and controlled access.
- **Ease of Use:** Provides a simple and intuitive graphical interface.
- **Efficiency:** Enables quick data retrieval and updates.
- **Cost-Effectiveness:** Built using open-source technologies, reducing implementation cost.

Project Significance

The system demonstrates how database management and software development concepts can be applied to solve real-world problems. It provides practical exposure to technologies such as database connectivity, authentication systems, and GUI development.

From an academic perspective, the project strengthens understanding of:

- Database management systems (DBMS)
- Secure authentication techniques
- Python programming and GUI design

From an organizational viewpoint, InvenTrack improves efficiency, reduces errors, and provides a reliable solution for managing inventory data. Its modular design allows future integration with advanced systems such as cloud platforms and enterprise applications.

1.2 Objectives of the Project

The primary objective of the InvenTrack system is to design and develop a secure, efficient, and user-friendly inventory management solution.

1. Automation:

To eliminate manual inventory tracking and automate data management processes.

2. Accuracy:

To ensure precise storage and retrieval of inventory data using a structured database system.

3. Security:

To implement secure login functionality using encrypted passwords and authentication mechanisms.

4. Efficiency:

To enable quick access, modification, and management of inventory records.

5. Usability:

To provide an intuitive graphical interface that can be easily used by non-technical users.

6. Scalability:

To design the system in a way that allows future enhancements such as cloud integration and multi-user access.

1.3 Relevance of the Project

In today's fast-paced digital environment, efficient inventory management is essential for the smooth functioning of organizations. Manual systems are not only time-consuming but also prone to errors and inefficiencies.

The InvenTrack system provides a modern solution by digitizing inventory management and ensuring secure data handling. It aligns with current technological trends, where businesses rely on software systems to manage operations effectively.

This project is also highly relevant academically, as it introduces students to real-world applications of:

- Database systems
- Secure authentication
- GUI development
- Software engineering principles

It bridges the gap between theoretical learning and practical implementation, preparing students for industry-level projects.

1.4 Scope of the Project

The InvenTrack system is designed as a desktop-based application with the following scope:

Current Scope:

- User authentication system
- Inventory data management
- Database connectivity
- GUI-based operations

Future Scope:

- Web-based or cloud-based system
- Advanced reporting and analytics
- Multi-user role management
- Integration with barcode scanning systems

The system is modular and scalable, allowing easy expansion in future versions.

1.5 Methodology Overview

The development of InvenTrack follows a structured methodology to ensure reliability and efficiency.

1. Requirement Analysis:

Understanding the need for an inventory management system and defining objectives.

2. System Design:

Designing architecture, database schema, and user interface.

3. Implementation:

Developing backend logic, database integration, and GUI.

4. Testing:

Testing individual modules and overall system functionality.

5. Deployment:

Setting up the system using the setup script and making it operational

CHAPTER -2

INTRODUCTION TO

TECHNOLOGY

2.1 Introduction to Python

Python is a high-level, interpreted, and general-purpose programming language developed by **Guido van Rossum** and first released in **1991**. Over the years, Python has become one of the most widely used programming languages due to its simplicity, readability, and versatility. It follows a design philosophy that emphasizes code readability and reduces the complexity of writing programs.

One of the key features of Python is that it is an **interpreted language**, meaning code is executed line by line. This makes debugging easier and speeds up development. Python is also **dynamically typed**, allowing developers to write code without explicitly defining variable types.

Python supports multiple programming paradigms such as:

- Object-Oriented Programming (OOP)
- Procedural Programming
- Functional Programming

Another major advantage of Python is its **extensive library support**, which allows developers to perform complex tasks using simple code. Libraries for database connectivity, GUI development, data analysis, and security make Python ideal for building real-world applications.

In the InvenTrack project, Python is used as the **core programming language** to implement:

- Database operations
- Authentication system
- GUI development
- Application logic

Its simplicity and flexibility make it an excellent choice for developing scalable and efficient applications.

2.2 Introduction to MySQL

MySQL is a popular open-source Relational Database Management System (RDBMS) used for storing and managing structured data. It is widely used in web and desktop applications due to its reliability, performance, and ease of use.

MySQL stores data in the form of tables consisting of rows and columns. It uses **Structured Query Language (SQL)** to perform operations such as:

- INSERT (adding data)
- SELECT (retrieving data)
- UPDATE (modifying data)
- DELETE (removing data)

In the InvenTrack system, MySQL plays a crucial role in:

- Storing user login credentials
- Managing inventory records
- Maintaining data consistency

Key Features of MySQL:

- High performance and speed
- Data integrity and consistency
- Support for large datasets
- Secure access control

Example Usage in Project:

The system connects to MySQL using:

```
import tkinter as tk
from tkinter import ttk, messagebox
import mysql.connector
from dotenv import load_dotenv
import os
import bcrypt
import random
import time
import re
```

This allows Python to execute SQL queries and interact with the database efficiently.

2.3 Introduction to Tkinter (GUI Library)

Tkinter is Python's standard library for building Graphical User Interfaces (GUI). It provides a set of tools and widgets that allow developers to create interactive desktop applications.

Tkinter is based on the Tk GUI toolkit and is included by default with Python, making it easy to use without additional installation.

Features of Tkinter:

- Simple and easy-to-use interface
- Cross-platform compatibility
- Lightweight and efficient
- Wide range of widgets

Widgets Used in the Project:

- Button
- Label
- Entry

- Frame
- Treeview

In the InvenTrack system, Tkinter is used to:

- Create login screens
- Display inventory data
- Provide navigation through buttons and menus
- Show messages and alerts

The GUI ensures that users can interact with the system easily without needing technical knowledge.

2.4 Introduction to bcrypt (Security Library)

bcrypt is a Python library used for secure password hashing. It is designed to protect passwords by converting them into encrypted formats that cannot be easily reversed.

Unlike simple hashing methods, bcrypt adds:

- **Salt (random data)**
- **Multiple hashing rounds**

This makes it extremely secure against attacks such as:

- Brute force attacks
- Dictionary attacks

Usage in the Project:

When a user registers or logs in:

- The password is hashed using bcrypt
- The hashed password is stored in the database
- During login, entered password is verified against the hash

Example:

```
import bcrypt

hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt())
```

Advantages:

- Strong security
- Protection against hacking
- Industry-standard authentication method

2.5 Introduction to python-dotenv

python-dotenv is a library used to manage environment variables in Python applications. It allows developers to store sensitive information such as database credentials in a **.env** file instead of hardcoding them in the source code.

Why It Is Important

- Keeps sensitive data secure
- Prevents exposure of credentials
- Makes configuration flexible

Usage in Project:

The system reads database credentials like:

- Host
- Username
- Password
- Database name

Example:

```
from dotenv import load_dotenv
import os
load_dotenv()
db_user = os.getenv("DB_USER")
```

This approach improves security and maintainability of the application.

2.6 Introduction to mysql.connector:

mysql.connector is a module that allows Python programs to connect and interact with MySQL databases.

It provides functions to:

- Establish database connection
- Execute SQL queries
- Fetch results

Example Connection:

```
conn = mysql.connector.connect(  
    host="localhost",  
    user="root",  
    password="password",  
    database="inventrack"  
)
```

Role in Project:

- Connects Python backend to MySQL
- Executes queries for inventory operations
- Handles data transactions

2.7 Supporting Technologies

1. Datetime Module:

Used to handle date and time operations in the system.

2. OS Module:

Used for file handling and system-level operations.

3. Setup Automation:

The setup.py script automates:

- Library installation
- Database setup
- Initial configuration

CHAPTER -3

SYSTEM DEVELOPMENT LIFE CYCLE (SDLC)

3.1 Introduction to SDLC

The **Software Development Life Cycle (SDLC)** is a structured process used for developing software in a systematic and organized manner. It consists of a sequence of steps that guide developers from the initial planning stage to the final deployment and maintenance of the software.

SDLC ensures that software is:

- Efficient
- Reliable
- Error-free
- Easy to maintain

For the **InvenTrack Inventory Management System**, SDLC helps in organizing development stages such as database design, authentication implementation, and GUI creation.

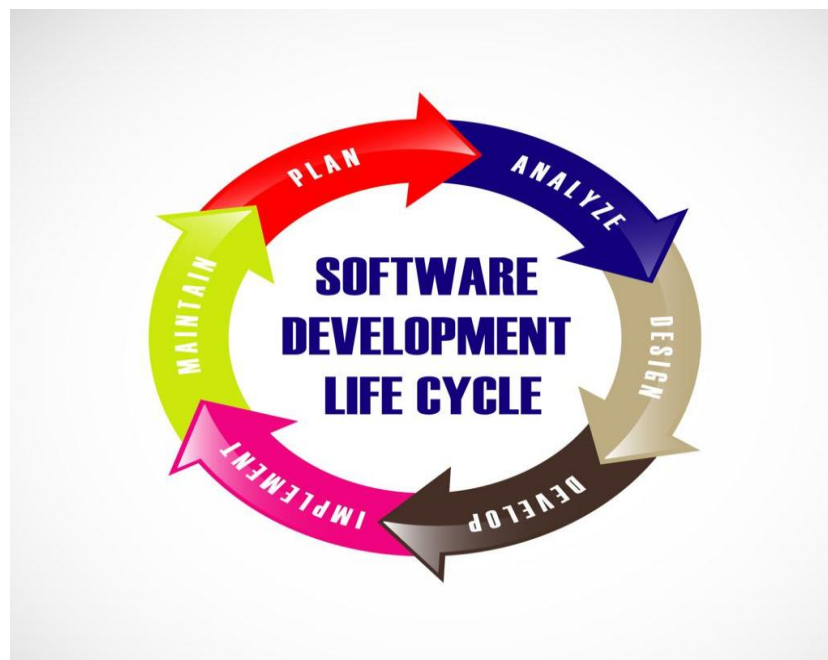


Fig 3.1 Software Development Life Cycle

The figure 3.1 shows the different steps of the Software Development Life Cycle in this figure we can see that firstly we do the planning of the project in which all the things related to the project is planned like the what is this project for, what is the objective of this software, for whom we are developing this software. In planning we do all this thing. Then in the next step we define all the planned things means in this step we gather all the information about the project and see the requirements we want in the software and in this step we gather all the requirement related to staff or all other things important in making a software. Let's we move to the third step this is the step in which we are design the software this work is mainly done by the web designers they do all the work related to the design then the coders code to get those output. Now, it's the time to do the real work means in this phase we do the coding all the coding is done in this phase and then we move to the next step of the SDLC which is testing phase. In this phase of SDLC we do all the testing work which is doing tests of the software or website using different methodologies of testing, different methods of tests are done in this phase. After all this work is done then the work of installing or deployment of the software is done or handover the software to the client.

3.2 Need of SDLC

The use of SDLC is very important in software development because:

- It provides a clear structure
- Helps in proper planning
- Reduces project failure
- Improves team coordination
- Ensures high-quality output

Without SDLC, development becomes unorganized. For example, if one developer starts coding without proper design, it may lead to errors and rework.

In the InvenTrack project, SDLC ensures:

- Proper planning of database
- Structured implementation of modules
- Smooth integration of GUI and backend

3.3 Phases of SDLC:

The **Software Development Life Cycle (SDLC)** is a structured methodology that divides the process of software development into multiple phases. Each phase has a specific purpose and contributes to the overall quality and efficiency of the system. In the development of the **InvenTrack Inventory Management System**, all SDLC phases were followed in a systematic manner to ensure that the final product is reliable, secure, and user-friendly.

3.3.1 Requirement Analysis Phase:

The Requirement Analysis phase is the foundation of the entire software development process. In this phase, the main objective is to understand the problem that needs to be solved and to gather all the necessary requirements of the system. This includes identifying the needs of users, defining the functionalities of the system, and understanding the environment in which the system will operate.

For the InvenTrack system, this phase involved analyzing the limitations of traditional inventory management methods, such as manual record-keeping and spreadsheet-based tracking. These methods were found to be inefficient, error-prone, and difficult to manage. Based on this analysis, the need for a secure, automated, and database-driven inventory management system was identified.

The requirements were categorized into functional and non-functional requirements. Functional requirements included features such as user authentication, inventory data management, and database operations. Non-functional requirements included aspects such as system security, performance, and usability. The outcome of this phase was a clear and detailed understanding of what the system should achieve.

3.3.2 Design Phase:

After the requirements were clearly defined, the next step was the Design phase. This phase focuses on creating a blueprint of the system before actual development begins. The design phase ensures that developers have a clear understanding of how the system will be structured and how different components will interact with each other.

In the InvenTrack project, the system design included the development of a three-layer architecture consisting of the presentation layer, logic layer, and data layer. The graphical user interface was designed using Tkinter, ensuring that it is simple, intuitive, and easy to navigate. The database structure was carefully planned using MySQL, where tables were created to store user credentials and inventory data in an organized manner.

The design phase also involved planning the flow of data between different modules, ensuring that the system operates smoothly without any conflicts. By creating a proper design before coding, the chances of errors during development were significantly reduced, and the overall development process became more efficient.

3.3.3 Implementation (Coding Phase):

The Implementation phase is the stage where the actual development of the system takes place. In this phase, the design created earlier is converted into a working system through coding. This is considered the most crucial and technically demanding phase of the SDLC.

In the InvenTrack system, Python was used as the primary programming language to develop the application. The backend logic was implemented to handle user authentication, database operations, and inventory management. The MySQL database was connected to the application using the `mysql.connector` module, enabling efficient storage and retrieval of data.

The graphical user interface was developed using Tkinter, which allowed users to interact with the system easily. Special attention was given to implementing secure authentication using `bcrypt`, ensuring that user passwords are encrypted and protected.

During this phase, several challenges were encountered, such as handling database connections and ensuring smooth interaction between the GUI and backend logic. These challenges were resolved through debugging, testing, and continuous improvement of the code. The successful completion of this phase resulted in a fully functional system.

3.3.4 Testing Phase:

Once the system was developed, it was necessary to test it thoroughly to ensure that it functions correctly and meets all the requirements. The Testing phase plays a critical role in identifying and fixing errors before the system is deployed.

In the InvenTrack project, different types of testing techniques were used to evaluate the system. Unit testing was performed to check individual components such as login functionality and database operations. Integration testing ensured that different modules worked together without any issues. Finally, system testing was conducted to verify the overall performance of the application.

Testing helped in identifying errors related to data handling, user input validation, and system performance. By resolving these issues, the reliability and efficiency of the system were significantly improved. This phase ensured that the final product is stable and ready for real-world use.

3.3.5 Deployment Phase:

After successful testing, the system was ready to be deployed. The Deployment phase involves making the software available for use by end users. This phase ensures that the system is properly installed and configured in the target environment.

In the case of InvenTrack, deployment was simplified through the use of a setup script. This script automates the installation of required libraries, configuration of the database, and initialization of the system. As a result, users can easily set up the application without requiring advanced technical knowledge.

The deployment phase ensured that the system is functional, accessible, and ready to be used in real-world scenarios.

3.3.6 Maintenance Phase:

The Maintenance phase is the final stage of the SDLC and continues even after the system has been deployed. In this phase, the system is monitored and updated to ensure that it continues to function efficiently.

For the InvenTrack system, maintenance includes fixing bugs, improving performance, and adding new features as required. As user needs evolve, the system can be updated to include additional functionalities such as advanced reporting or cloud integration.

Maintenance is essential for ensuring the long-term success and usability of the system. It allows the software to adapt to changing requirements and technological advancements

3.4 DIFFERENT TYPES OF SDLC MODELS:

There are several SDLC models used in software development:

- Waterfall Model
- Incremental Model
- Spiral Model
- V-Model

Each model is selected based on project requirements.

3.5 Waterfall Model

The Waterfall model is one of the classical models of the Software Development Life Cycle (SDLC). In this model, the steps of development are followed in a **linear** and **sequential** manner, where each phase must be completed before moving to the next. The process begins with planning and requirement analysis, followed by system design, implementation (coding), testing, deployment, and maintenance. Unlike iterative models, the Waterfall approach **does not repeat phases**; once a stage is finished, you cannot go back. This model is best suited for **simple, well-defined, and stable projects** where requirements do not change frequently. It is easy to understand, well-structured, and provides clear documentation at every phase, making it an efficient method for developing software with fixed requirements.

Features of Waterfall Model

- Simple and easy to understand
- Well-structured and disciplined approach
- Clear documentation at every stage

- Easy to manage because each phase has clear deliverables and deadlines

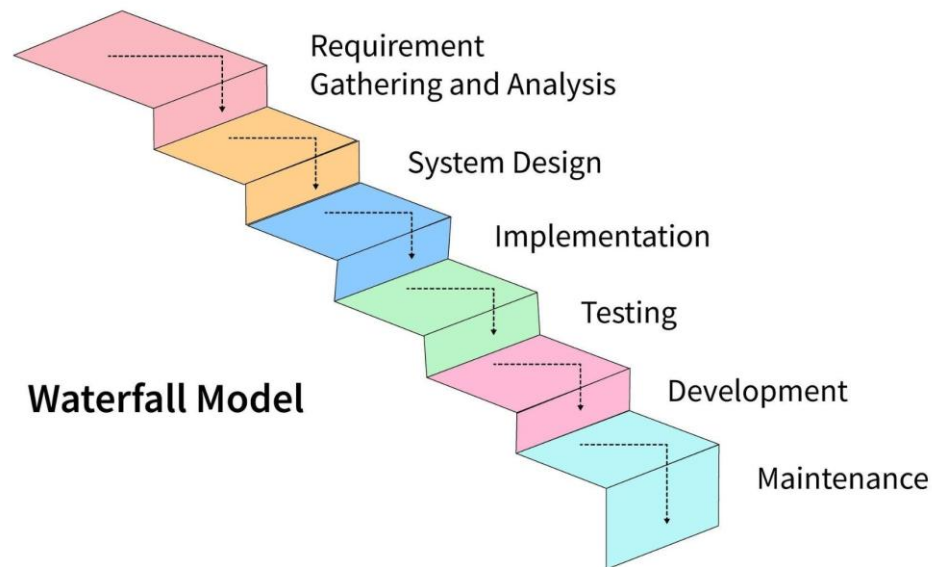


Fig 3.2 Waterfall model

The above figure shows the representation of waterfall model while developing a software.

CHAPTER -4

SYSTEM ANALYSIS

4.1 Problem Definition:

Traditional inventory management systems suffer from several limitations, including data redundancy, inaccuracies, and inefficient handling of records. Manual methods such as maintaining registers or spreadsheets often lead to duplication of data, human errors, and difficulty in tracking inventory status in real time.

In many cases, there is also a lack of proper security, as unauthorized users may access or modify inventory data. Additionally, manual systems are time-consuming and do not provide quick access to information when needed.

The proposed InvenTrack system resolves these drawbacks by providing a secure, automated, and database-driven solution. It ensures accurate data storage, controlled user access through authentication, and efficient management of inventory records using a graphical interface.

4.2 Feasibility Study:

a) Technical Feasibility

The system is developed using Python, which is an open-source and widely supported programming language. The use of MySQL ensures efficient and reliable database management, while Tkinter provides a simple and effective graphical user interface.

The project runs smoothly on standard systems with moderate hardware configurations, making it accessible without requiring high-end resources. All technologies used in the system are cross-platform, ensuring easy deployment on different operating systems.

b) Economic Feasibility

The InvenTrack system is cost-effective as it is built entirely using open-source technologies. There is no need for expensive software or proprietary tools. The system can be implemented using existing computer infrastructure, which reduces the overall cost of development and deployment.

c) Operational Feasibility

The system is designed with a user-friendly interface that allows users to operate it with minimal training. The GUI provides clear navigation and real-time feedback, making it easy for users to manage inventory data efficiently.

The system improves operational efficiency by reducing manual effort and ensuring accurate record-keeping.

d) Schedule Feasibility

The development of the InvenTrack system was completed within the academic timeline. The project was developed in a structured manner over a period of six weeks:

- Week 1 – Requirement Analysis
- Week 2–3 – System Design
- Week 4 – Implementation (Coding)
- Week 5 – Testing
- Week 6 – Documentation

This structured schedule ensured timely completion of the project.

4.3 Hardware and Software Requirements

The successful implementation of the InvenTrack system requires the following hardware and software components:

Component	Specification
Processor	Intel Core i3 / Ryzen 3 or higher
RAM	Minimum 4 GB (8 GB recommended)
Storage	At least 500 MB free disk space
Operating System	Windows 10 / Linux / macOS

Component	Specification
Programming Language	Python 3.8 or above
Database	MySQL
IDE	VS Code / PyCharm / IDLE
Libraries	mysql.connector, tkinter, bcrypt, python-dotenv

4.4 Functional Requirements

The functional requirements define the operations that the system must perform.

1. The system must allow secure user login and authentication.
2. The system must store and manage inventory data in a database.
3. The system must support CRUD operations (Create, Read, Update, Delete).
4. The system must display inventory data through a graphical interface.
5. The system must connect and interact with the MySQL database.
6. The system must allow users to update and delete records as required.

4.5 Non-Functional Requirements

Attribute	Description
Performance	Fast data retrieval and processing
Security	Encrypted passwords and secure login
Usability	GUI operable by non-technical users

Attribute	Description
Reliability	Accurate and consistent data handling
Maintainability	Easy updates and modifications
Scalability	Capable of handling large datasets

4.6 Identification of Needs

The development of the InvenTrack system is based on the following needs:

- Centralized storage of inventory data for better management
- Secure access control to prevent unauthorized usage
- Faster processing and retrieval of data
- Efficient report generation for decision-making
- Ability to integrate with advanced systems in the future

CHAPTER -5

SYSTEM DESIGN

5.1 Overview:

System Design translates the requirements gathered during the analysis phase into a structured technical blueprint that guides the development of the system. It defines how different components of the system will interact and ensures that data flows smoothly between modules in a consistent and efficient manner. The design phase plays a crucial role in improving system performance, reliability, and maintainability.

For the **InvenTrack Inventory Management System**, the design focuses on creating a well-organized architecture that connects the graphical user interface, processing logic, and database. The system is designed to ensure fast data processing, secure user authentication, and efficient handling of inventory records.

5.2 System Architecture:

a) Layered Architecture:

The InvenTrack system follows a layered architecture that divides the system into three major components. This separation helps in improving scalability, maintainability, and ease of development.

1. Presentation Layer (GUI):

This layer is responsible for all user interactions with the system. It provides a graphical interface through which users can access various features of the application. The interface is designed using Tkinter, ensuring simplicity and ease of use.

The main functions of this layer include displaying the login screen and dashboard, accepting user inputs, and presenting inventory data in a structured format. It also provides buttons, menus, and status messages that guide the user through different operations such as adding, updating, deleting, and viewing inventory records.

2. Logic Layer (Processing Engine):

The logic layer acts as the core of the system, where all processing and decision-making operations take place. It connects the user

interface with the database and ensures that all user requests are handled correctly.

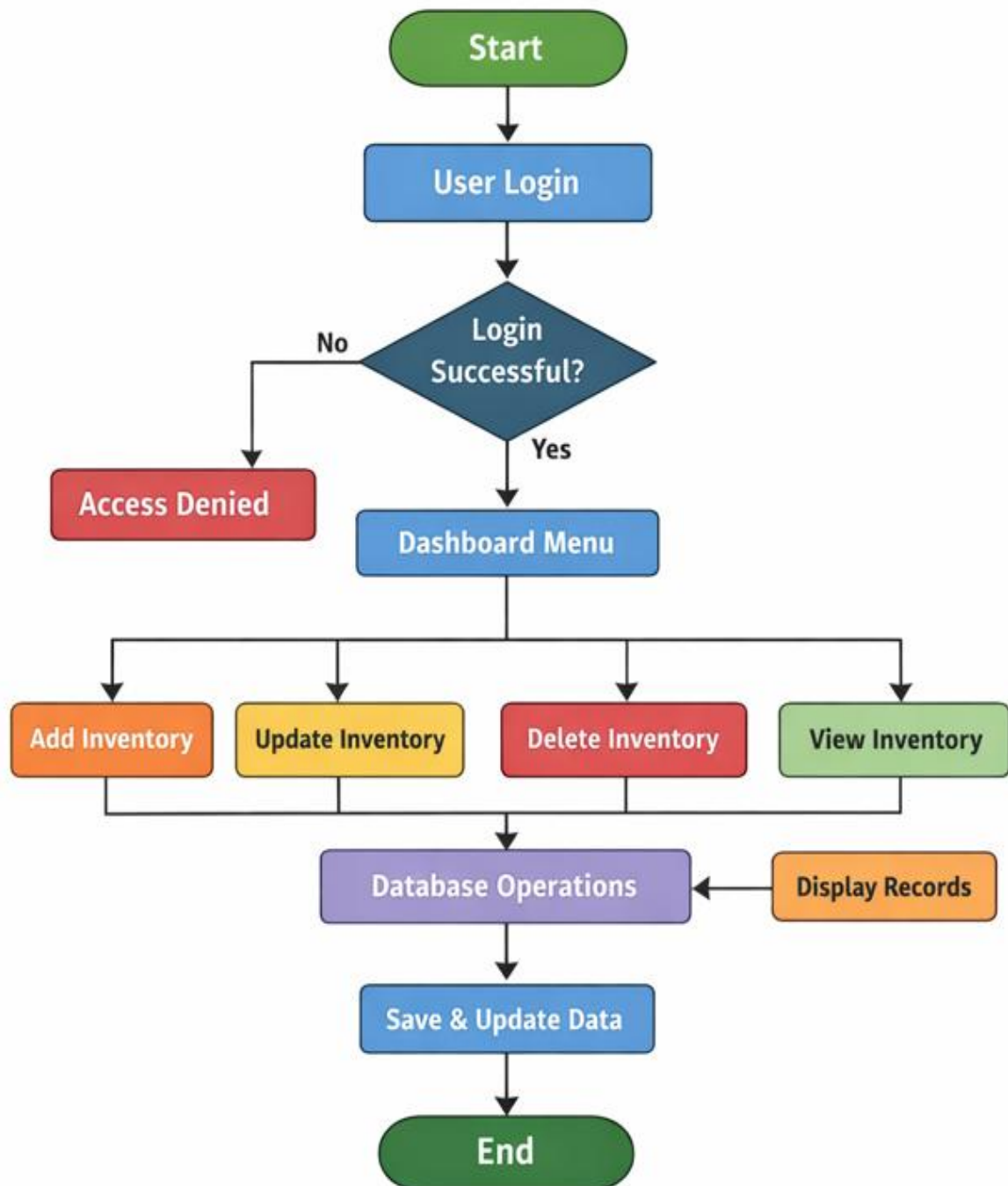
This layer is responsible for validating user inputs, handling authentication processes, and executing inventory-related operations. It processes data received from the GUI and performs operations such as inserting new records, updating existing data, retrieving information, and deleting records from the database. After processing, the results are sent back to the presentation layer for display.

3. Data Layer (Storage):

The data layer manages all data-related operations in the system. It is responsible for storing, retrieving, and maintaining data in a structured format using a MySQL database.

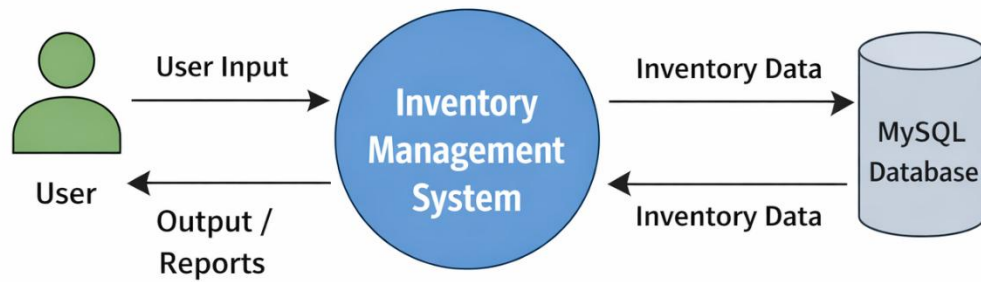
This layer stores user information such as login credentials and inventory details such as item name, quantity, and category. It ensures data consistency, security, and efficient retrieval of records. All database queries are executed through this layer, making it a critical component of the system architecture.

b) Process Flow Diagram (Descriptive)

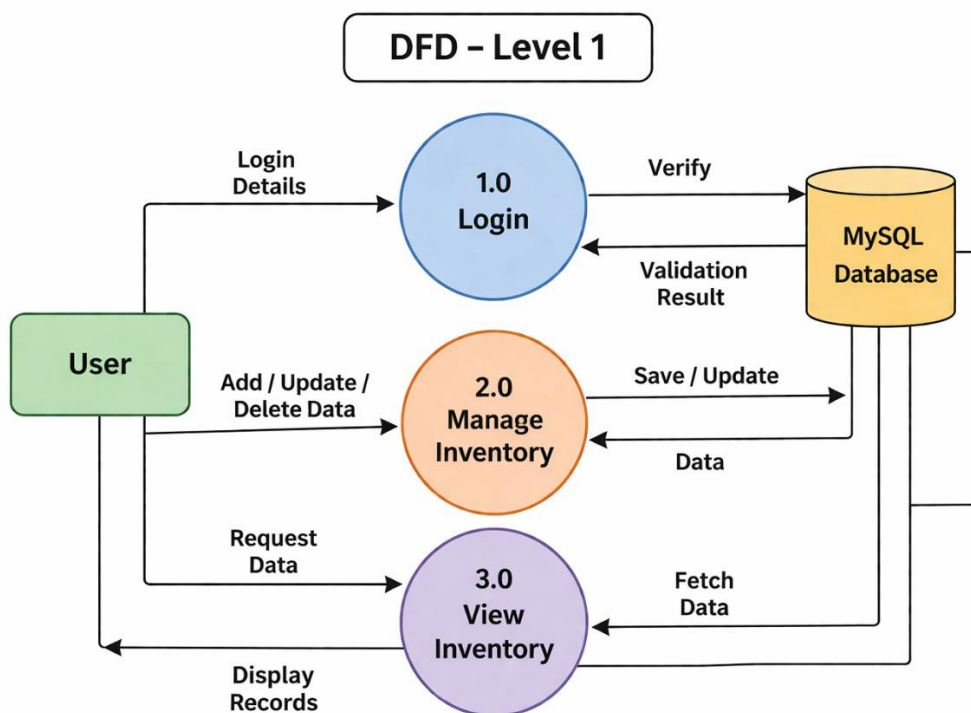


5.3 Data Flow Diagram

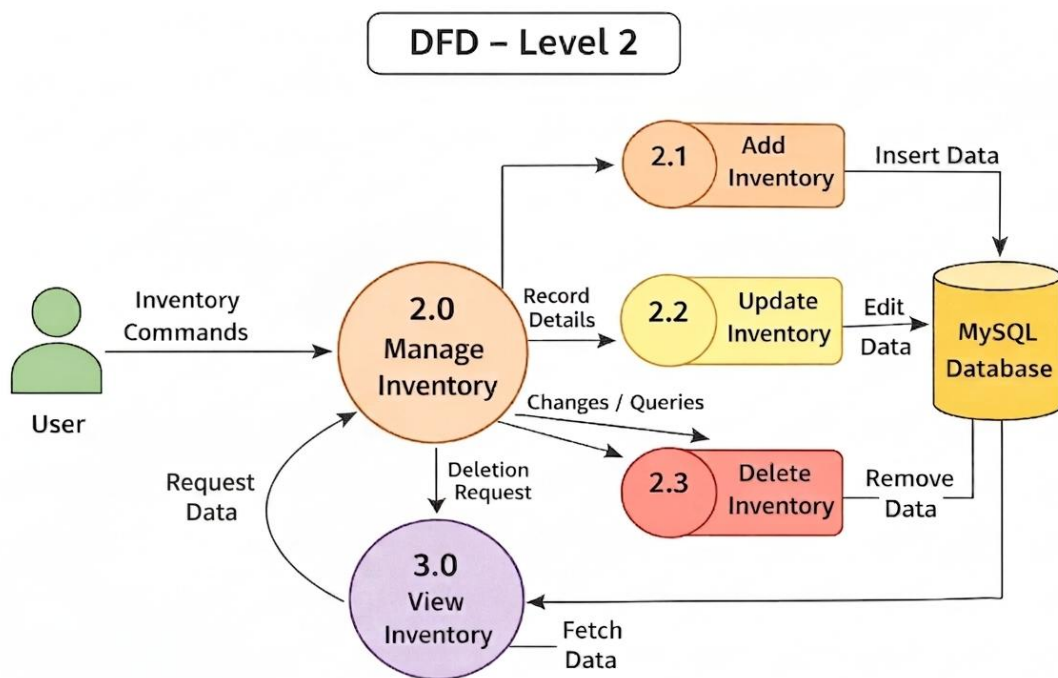
5.3.1 Level 0



5.3.2 Level 1



5.3.3 Level 2



5.4 Database Design:

A structured SQL database is used in the InvenTrack system to store and manage data efficiently. The database follows relational schema principles and consists of multiple interconnected tables. These tables are created automatically during the setup process using the setup script.

Entities:

1. Users

- User ID : string (unique)
- Name : string
- Email : string (unique)
- Password : string (hashed using bcrypt)
- Role : enum ('admin', 'user')
- is_deleted : Boolean

2. Categories

- Category ID : integer (auto-increment, unique)
- Category Name : string (unique)
- Description : string

3. Suppliers

- Supplier ID : integer (auto-increment, unique)
- Supplier Name : string
- Contact Name : string
- Phone : string
- Email : string
- Address : string
- is_deleted : Boolean

4. Product

- Product ID : string (unique)
- Product Name : string
- SKU : string (unique)
- Price : float
- Cost Price : float
- Stock : integer
- Reorder Level : integer
- Category ID : integer (foreign key)
- Supplier ID : integer (foreign key)
- is_deleted : boolean

5. Orders

- Order ID : integer (auto-increment, unique)
- User ID : string (foreign key)
- Product ID : string (foreign key)
- Quantity : integer
- Order Status : enum ('placed', 'shipped', 'delivered', 'cancelled', 'refunded')
- Created At : timestamp

6. Payment

- Payment ID : integer (auto-increment, unique)
- Order ID : integer (foreign key, unique)
- Amount : float
- Payment Status : enum ('pending', 'completed', 'failed', 'refunded')
- Paid At : timestamp

Relationships:

- Product is linked with Categories using Category ID
- Product is linked with Suppliers using Supplier ID
- Orders are linked with Users using User ID
- Orders are linked with Product using Product ID
- Payment is linked with Orders using Order ID

The data is stored in MySQL tables and managed through SQL queries executed by the application. This design ensures efficient data handling, maintains data integrity, and supports all operations of the InvenTrack system.

5.5 Algorithm Design

Algorithm 1: User Login and Authentication

- Step 1 Initialize database connection and load user data.
- Step 2 Prompt user to enter email and password through GUI.
- Step 3 Retrieve stored hashed password from database using email.
- Step 4 Compare entered password with stored hash using bcrypt.
- Step 5 If credentials match, grant access to the system.
- Step 6 If credentials do not match, display error message.
- Step 7 Redirect user to dashboard upon successful login.

Algorithm 2: Inventory Management (Add / Update / Delete)

Step 1: Initialize database connection.

Step 2: Accept user input for inventory details (item name, quantity, category, etc.).

Step 3: Validate input fields to ensure no empty or invalid values.

Step 4: Based on operation selected:

- Add → Insert new record into database.
- Update → Modify existing record using item ID.
- Delete → Remove or mark record as deleted.

Step 5: Execute corresponding SQL query.

Step 6: Commit changes to database.

Step 7: Display confirmation message on GUI.

Algorithm 3: View Inventory Records

Step 1: Establish database connection.

Step 2: Fetch all inventory records using SELECT query.

Step 3: Store retrieved data in a structured format.

Step 4: Display records in GUI table (Treeview).

Step 5: Allow user to refresh or search records if required.

Algorithm 4: System Setup and Initialization

Step 1: Run setup script.

Step 2: Install required Python libraries.

Step 3: Create database and required tables.

Step 4: Insert default admin user into Users table.

Step 5: Create environment configuration file (.env).

Step 6: Display setup completion message.

5.6 Interface Design

The interface of the InvenTrack system is designed to provide smooth interaction between the user and the application. It ensures that users can easily perform operations such as login, data entry, and inventory management without requiring technical expertise.

Input Interfaces

- Login form to accept user credentials (email and password).
- Text fields for entering inventory details such as item name, quantity, and category.

Output Interfaces

- Inventory records displayed in tabular format using GUI components.
- Status messages to indicate successful or failed operations.
- Confirmation dialogs for actions such as update or deletion.

5.7 Error Handling and Validation

Proper error handling and validation mechanisms are implemented to ensure smooth functioning of the system and to prevent invalid operations.

- Invalid login credentials → Display error message.
- Empty input fields → Prompt user to fill required details.
- Duplicate entries → Warn user and prevent insertion.
- Database connection failure → Display error and retry option.
- Invalid data types → Show validation message to correct input.

5.8 User Interface Navigation

The InvenTrack system provides a simple and intuitive navigation structure through its main window. The interface is designed to allow users to access all functionalities easily.

Main Window → Options:

- Login
- Add Inventory
- Update Inventory
- Delete Inventory
- View Records
- Exit

Each option is connected to a specific function, and clicking a button opens the corresponding interface or performs the required operation.

5.9 Design Considerations

Several important factors were considered while designing the system to ensure usability and efficiency.

- Layout is organized using frames and grid structure for consistency across different screen sizes.
- Event handling is implemented using command parameters to simplify user interactions.
- Input validation ensures data accuracy and prevents errors.
- Font sizes and colors are selected carefully to enhance readability and user experience.

CHAPTER -6

SCREEN DESIGN

6.1 Importance of Screen Design

A well-structured screen layout ensures smooth interaction between the user and the system. Good screen design reduces user confusion, minimizes errors, and improves efficiency while performing routine operations. Since inventory management tasks are performed frequently, it is essential that the interface remains clear, responsive, and easy to use.

The design of the InvenTrack system focuses on simplicity and usability so that even non-technical users can operate it without difficulty. The interface is designed to reduce unnecessary input and provide quick access to important functions.

The design emphasizes:

- Minimal user input for faster operation
- Clearly visible buttons for key actions
- Organized display of inventory data
- Instant status and confirmation messages

6.2 GUI Architecture

The graphical user interface of the InvenTrack system is divided into three main components to ensure proper organization and usability.

- **Dashboard Frame** – acts as the main navigation panel from where users can access all functionalities.
- **Form/Input Frame** – used for entering and updating inventory data.
- **Display Frame** – shows inventory records and system outputs in tabular format.

The interface is built using Tkinter, and its grid layout manager is used to arrange widgets in a structured manner. This ensures that the layout remains consistent and properly aligned across different screen sizes and resolutions.

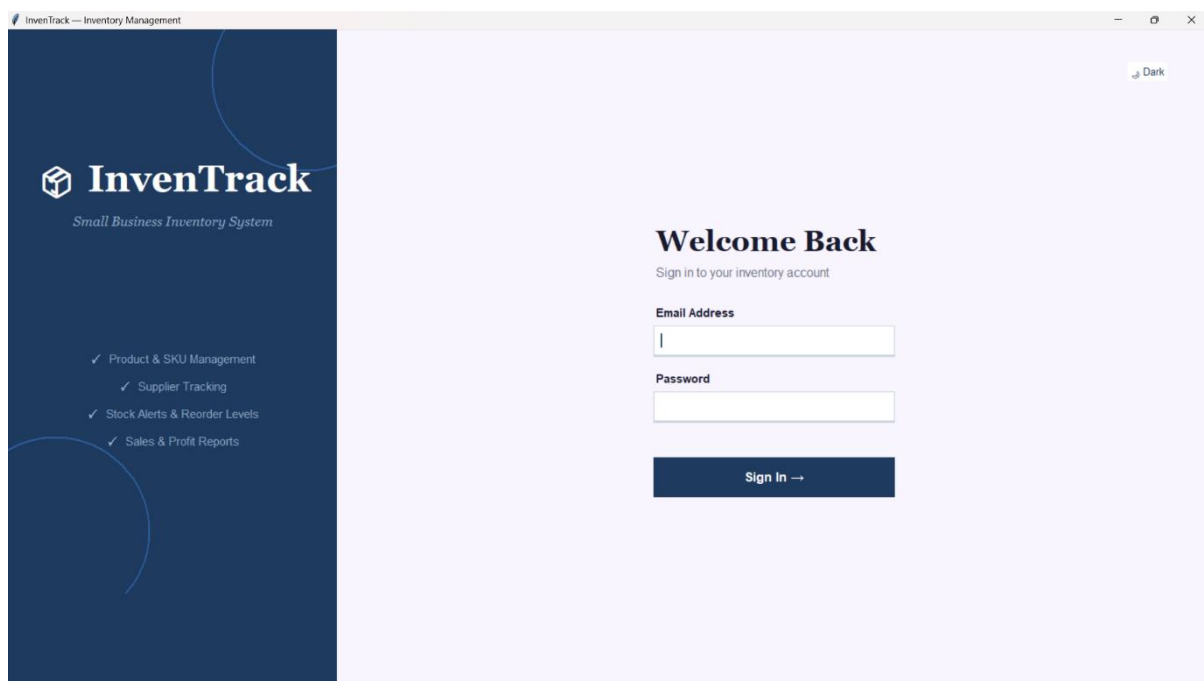
6.3 Key Screens

a) Login Screen

- Displays system branding (“InvenTrack”) with feature highlights
- Input fields: Email and Password
- Button: “Sign In”

Workflow:

1. User enters login credentials.
2. System validates input and checks database records.
3. Password is verified using bcrypt hashing.
4. On success, user is redirected to the main dashboard.
5. On failure, error message with attempt count is displayed.

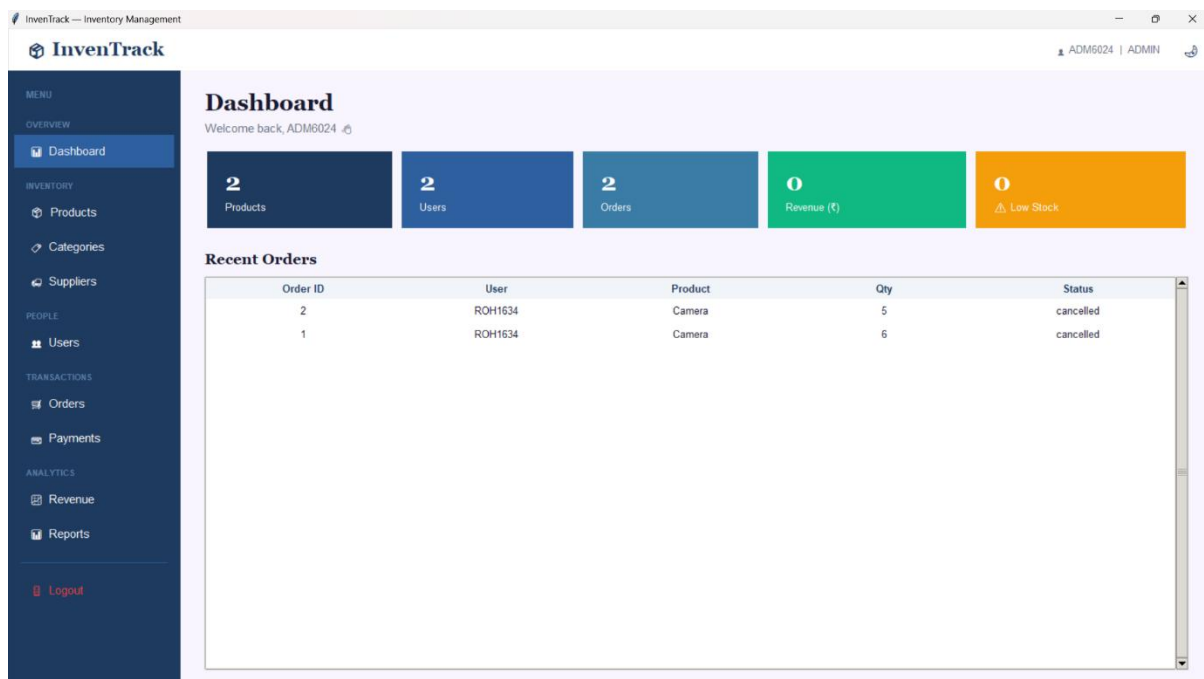


b) Dashboard Screen:

The dashboard acts as the central overview panel of the system.

- Displays welcome message with user ID
- Shows summary cards:
 - o Total Products
 - o Total Users
 - o Total Orders
 - o Revenue
 - o Low Stock Alerts
- Displays recent orders in a table format

This screen is implemented using stat cards and table views for quick data visualization.

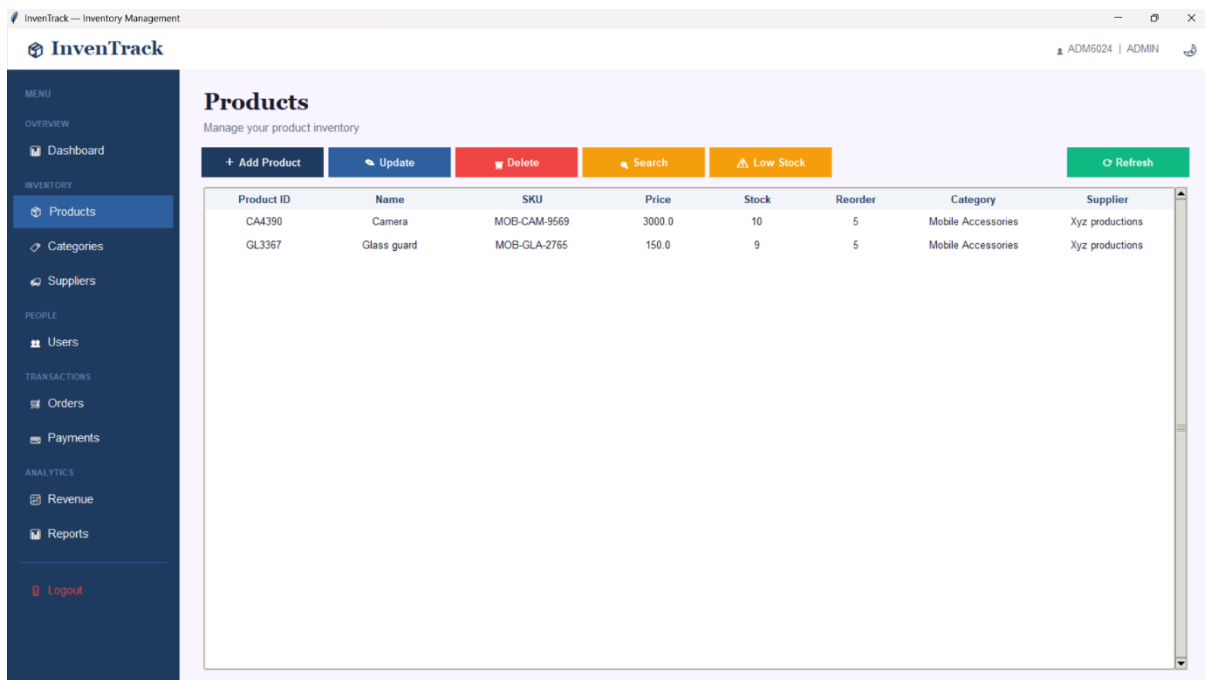


c) Products Management Screen:

This screen allows full control over inventory items.

Features:

1. Add Product
 2. Update Product
 3. Delete Product
 4. Search Product
 5. Low Stock Filter
 6. Refresh Data
- Data is displayed using `ttk.Treeview()` table
 - Low stock and out-of-stock items are highlighted with colors



InvenTrack — Inventory Management

Products
Manage your product inventory

[+ Add Product](#) [Update](#) [Delete](#) [Search](#) [Low Stock](#) [Refresh](#)

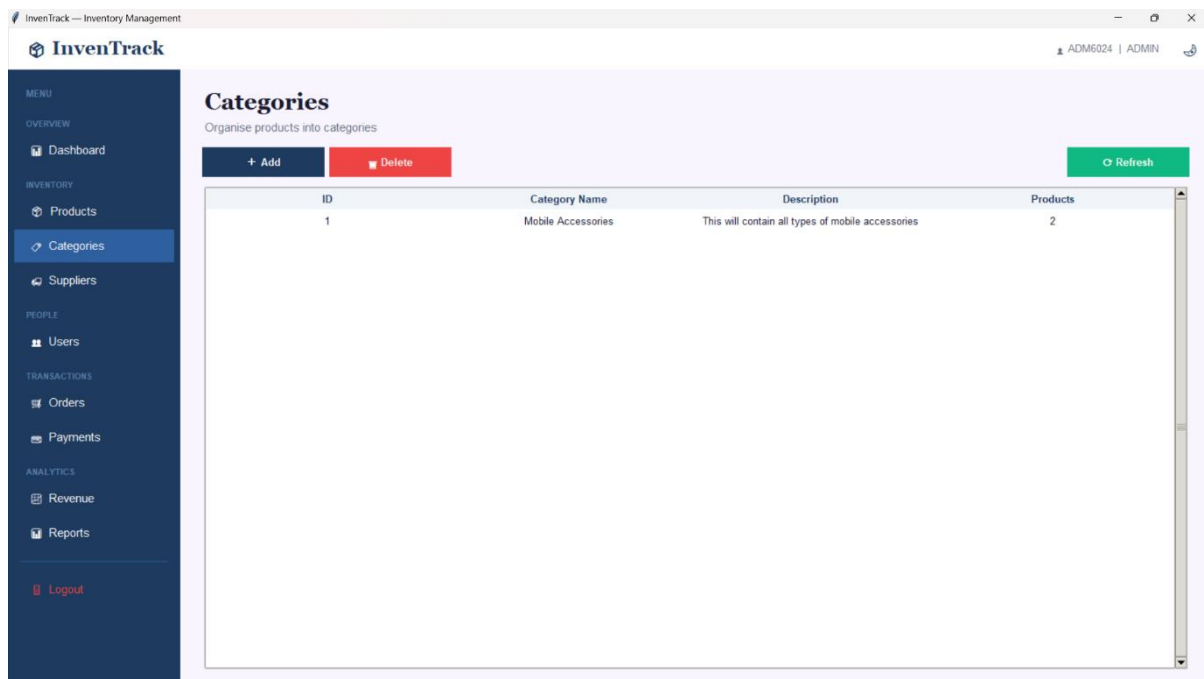
Product ID	Name	SKU	Price	Stock	Reorder	Category	Supplier
CA4390	Camera	MOB-CAM-9569	3000.0	10	5	Mobile Accessories	Xyz productions
GL3367	Glass guard	MOB-GLA-2765	150.0	9	5	Mobile Accessories	Xyz productions

d) Categories Screen:

This screen is used to manage product categories.

- Add new category
- Delete category
- View category list with product count

It ensures proper organization of products within the system.



e) Suppliers Screen:

This screen manages supplier information.

- Add supplier details
- Update supplier data
- Delete supplier
- View supplier-product relationships

InvenTrack — Inventory Management

ADM6024 | ADMIN

Suppliers

Manage your product suppliers

+ Add Supplier Update Delete Refresh

ID	Company	Contact	Phone	Email	Products
1	Xyz productions	Kuldeep	0394738291	kuldeep@gmail.com	2

f) Users Screen:

This module manages system users.

- Add new user (with role: admin/user)
- Update email
- Delete user
- View all users

User authentication and role-based access are handled through this screen.

InvenTrack — Inventory Management

InvenTrack

ADM6024 | ADMIN

Users
Manage system users

+ Add User Update Email Delete Refresh

User ID	Name	Email	Role
ADM6024	admin	admin@gmail.com	admin
ROH1634	Rohan	rohan@gmail.com	user

g) Orders Screen:

This screen allows administrators to manage all orders.

- View all orders
- Change order status (placed, shipped, delivered, cancelled)

InvenTrack — Inventory Management

ADM5024 | ADMIN

Orders

View and manage all orders

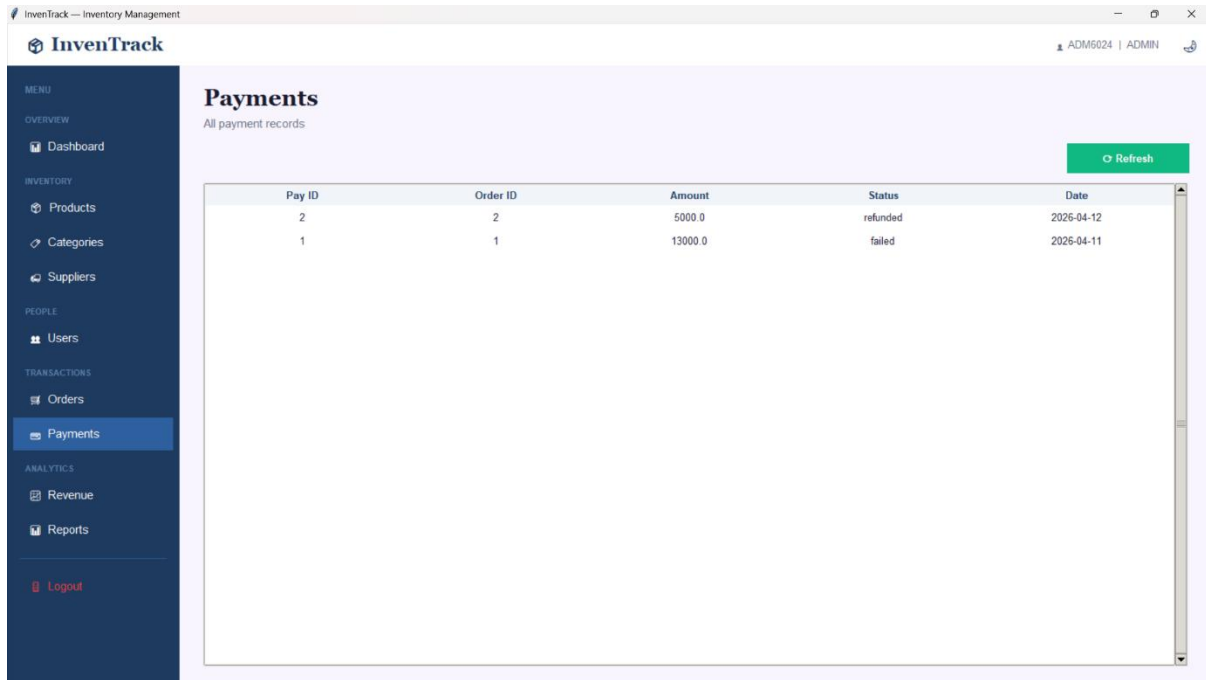
[Change Status](#) [Refresh](#)

Order ID	User	Product	Qty	Status	Date
2	ROH1634	Camera	5	cancelled	2026-04-12
1	ROH1634	Camera	6	cancelled	2026-04-11

h) Payments Screen:

This module handles payment records.

- View payment details
- Track payment status (pending, completed, refunded)



The screenshot displays the 'Payments' screen in the InvenTrack application. The interface includes a dark blue sidebar with a menu, a top header with the application name and user information, and a main content area. The 'Payments' section is titled 'Payments' with the subtitle 'All payment records'. A green 'Refresh' button is located in the top right corner of the main content area. The payment records are listed in a table with the following columns: Pay ID, Order ID, Amount, Status, and Date.

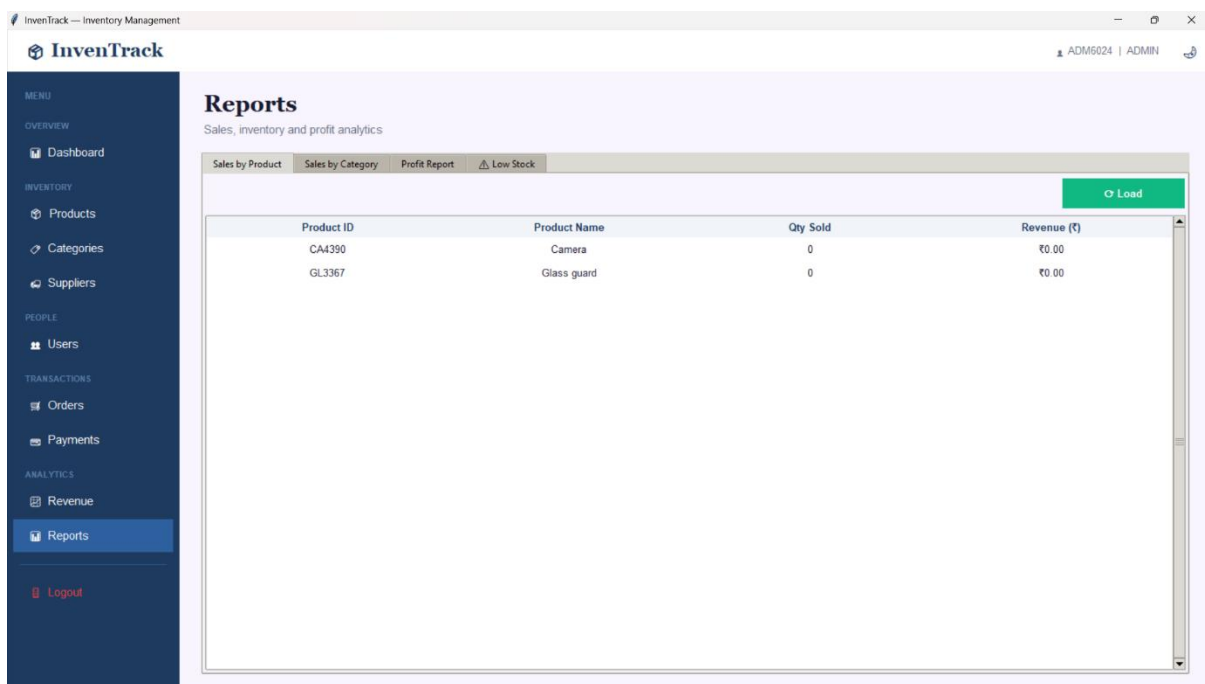
Pay ID	Order ID	Amount	Status	Date
2	2	5000.0	refunded	2025-04-12
1	1	13000.0	failed	2025-04-11

i) Reports & Analytics Screen:

This screen provides analytical insights.

- Sales by product
- Sales by category
- Profit report
- Low stock report

Data is displayed using tab-based interface for better organization.



The screenshot shows the 'InvenTrack — Inventory Management' application. The 'Reports' section is active, displaying 'Sales, inventory and profit analytics'. A tabbed interface at the top includes 'Sales by Product', 'Sales by Category', 'Profit Report', and 'Low Stock'. The 'Sales by Product' tab is selected, showing a table with the following data:

Product ID	Product Name	Qty Sold	Revenue (₹)
CA4390	Camera	0	₹0.00
GL3367	Glass guard	0	₹0.00

A 'Load' button is visible in the top right corner of the table area. The left sidebar contains navigation links for Overview, Inventory, People, Transactions, and Analytics, with 'Reports' currently selected.

6.4 Screen Design Principles Followed

- **Consistency:** Uniform color themes (light/dark) and layout across all screens
- **Feedback:** Message boxes for success, error, and confirmation
- **Accessibility:** Clear fonts, spacing, and readable UI elements
- **Simplicity:** Sidebar-based navigation reduces complexity
- **Responsiveness:** Dynamic content frame updates without reloading entire UI
- **Error Prevention:** Input validation and restricted actions (e.g., preventing invalid deletions)

CHAPTER -7

BACKEND AND CONNECTIVITY

7.1 Overview

The backend of the InvenTrack system is responsible for handling all core operations such as user authentication, inventory management, data processing, and database communication. It acts as a bridge between the graphical user interface (GUI) and the database layer, ensuring that user actions are correctly processed and stored.

The system is developed using Python and integrates database connectivity to manage inventory records efficiently. The backend processes user inputs received from the interface, performs validation, executes database queries, and returns appropriate responses to the GUI. This ensures smooth functioning of all modules such as product management, user management, and reporting.

The modular design of the backend improves maintainability and allows future enhancements such as cloud integration or API-based services.

7.2 Module-Wise Description

The backend is divided into several modules, each responsible for specific functionality within the system.

1. Authentication Module

This module handles user login and access control. It verifies user credentials by comparing the entered password with the stored hashed password in the database. Secure hashing techniques such as bcrypt are used to ensure data security.

2. Inventory Management Module

This module is responsible for managing all inventory-related operations. It allows users to add new items, update existing records, delete products, and retrieve inventory details from the database.

3. Category and Supplier Module

This module manages classification and organization of inventory items. It allows the creation, updating, and deletion of categories and supplier information, ensuring better data organization.

4. Orders and Transactions Module

This module handles order processing and tracks transaction details. It maintains records of orders, updates their status, and manages payment-related information.

5. Reporting and Analytics Module

This module generates reports based on inventory data, such as stock availability, sales trends, and low stock alerts. It helps administrators make informed decisions.

6. GUI Control Module

This module connects the frontend interface with backend logic. It ensures that user actions such as button clicks trigger the appropriate backend operations and display results on the screen.

7.3 Data Persistence Using Database

The InvenTrack system uses a relational database (such as MySQL) to store and manage data in a structured format. Unlike file-based storage, a database provides better scalability, security, and data integrity.

Advantages

- Structured storage of data in tables
- Faster data retrieval using queries
- Supports relationships between entities
- Ensures data consistency and integrity

Database Operations

- INSERT → Add new records
- SELECT → Retrieve data
- UPDATE → Modify existing records
- DELETE → Remove records

The database connection is established using Python database connectors, enabling seamless communication between the application and the database.

7.4 File Structure

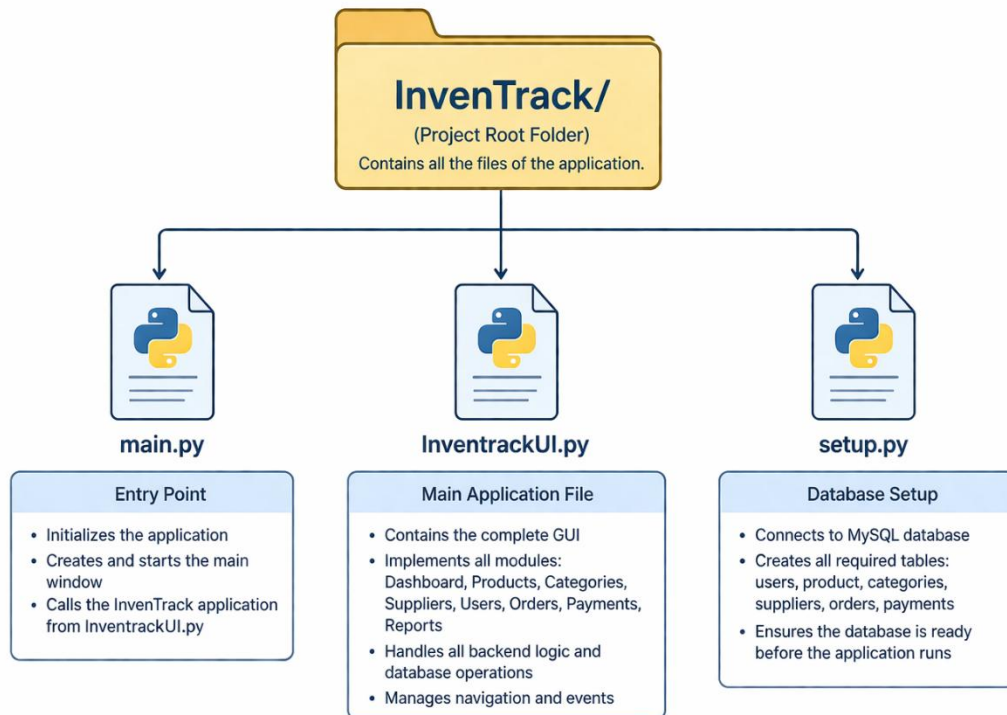


Fig 7.1: File Structure of InvenTrack Project

The project contains three main files in a single folder. All functionalities are implemented within these files.

7.5 Database Connectivity

The system connects to the database using Python's database connector library.

- Connection is established using host, username, password, and database name
- Queries are executed using cursor objects
- Transactions are committed to save changes

This ensures that all operations such as adding inventory, updating records, and retrieving data are performed reliably.

7.6 Security Measures

To ensure system security and data protection, the following measures are implemented:

- Passwords are stored in hashed format using bcrypt
- Input validation prevents invalid or malicious data
- Database queries are handled securely to avoid injection attacks
- Role-based access control restricts unauthorized actions
- Error handling prevents system crashes and data loss

CHAPTER -8

SOFTWARE TESTING

8.1 Testing Objective

The primary objective of testing the InvenTrack system is to ensure that each component performs as intended and that the integrated system meets user requirements in terms of accuracy, reliability, and usability. The testing process validates that all modules such as inventory management, user authentication, and data handling operate correctly under normal working conditions.

The goal is to identify errors, ensure smooth interaction between modules, and verify that the system produces correct outputs for given inputs.

8.2 Testing Strategy

The system follows a **Black Box Testing** approach, where testing is performed based on inputs and expected outputs without analyzing internal code structure.

Testing was performed manually during development and includes the following levels:

- 1) Unit Testing
- 2) Integration Testing
- 3) User Acceptance Testing

This strategy ensures that both individual components and the complete system function properly.

8.3 Unit Testing:

In this phase, individual functions and modules of the system were tested independently to ensure correctness. **Examples:**

Function	Input	Expected Output	Result
login_user()	Valid credentials	Login successful	Pass
add_product()	Valid product data	Record inserted	Pass
update_product()	Modified data	Record updated	Pass
delete_product()	Selected item	Record removed	Pass

Each function was tested multiple times with different inputs to ensure reliability and consistency.

8.4 Integration Testing

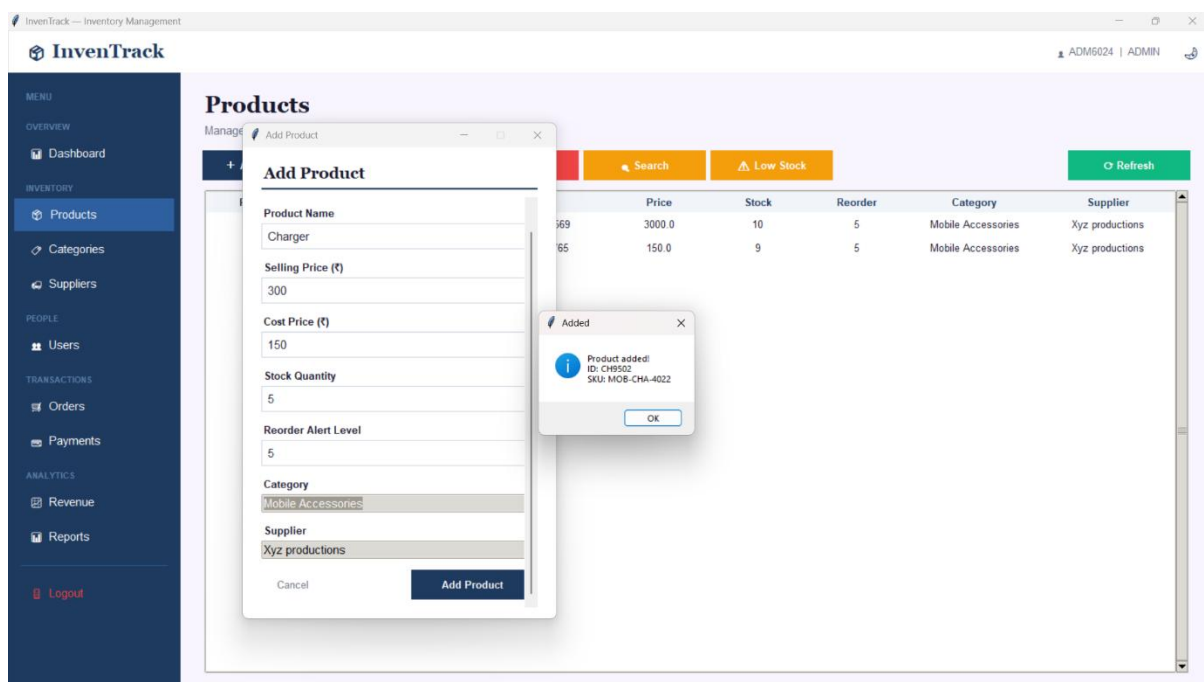
Integration testing ensures that different modules of the system work together without errors or data loss.

Test Flow:

User Input → Processing → Database Update → Display Output

This confirms that:

- Data entered through GUI is correctly processed
- Database operations are executed properly
- Output is displayed accurately



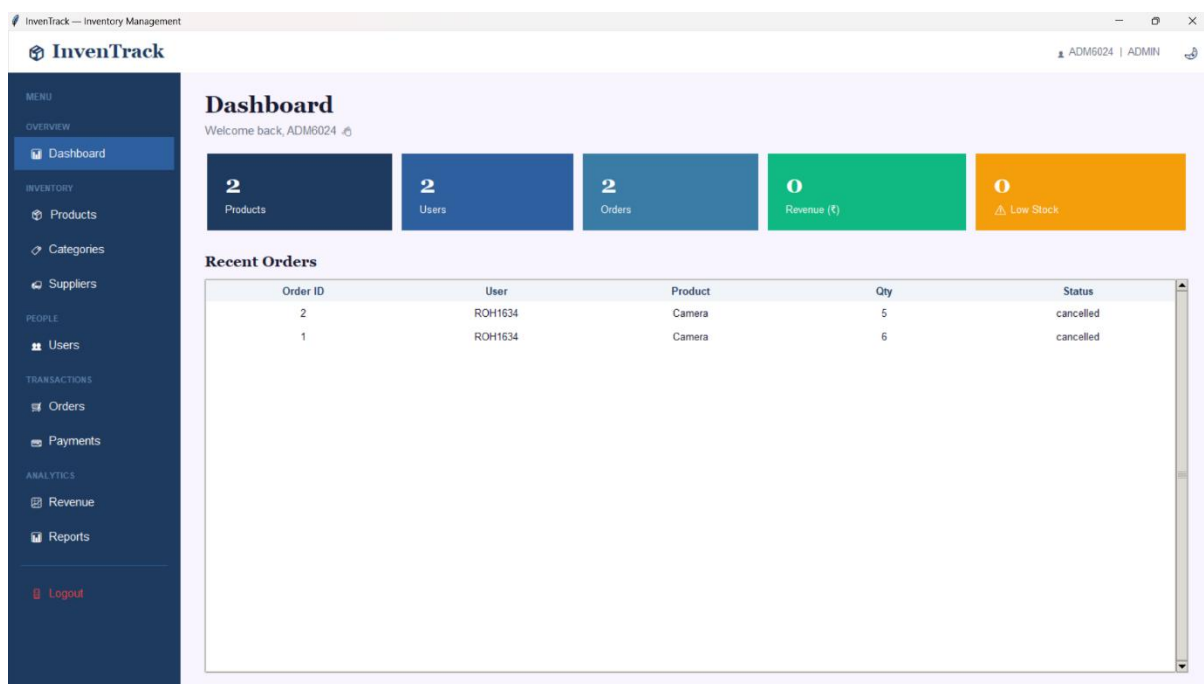
8.5 Functional Testing

Functional testing verifies that all features of the system operate according to requirements.

Checklist:

- Buttons trigger correct operations
- Data is stored and retrieved correctly
- GUI updates after each action
- Navigation between screens works properly

For example, when a new product is added, it immediately appears in the inventory list, confirming successful execution.



8.6 Error Handling Testing:

The system was tested for handling various error conditions to ensure robustness.

- Invalid login credentials → Error message displayed
- Empty input fields → Warning prompt shown
- Duplicate entries → Prevented by validation

- Database errors → Handled gracefully

Welcome Back

Sign in to your inventory account

Email Address

admin@gmail.com

Password

•••••

Invalid credentials. 2 attempt(s) left.

Sign In →

8.7 Sample Test Cases:

Test ID	Scenario	Expected Result	Status
TC-01	Invalid Login	Error message displayed	Pass
TC-02	Add New Product	Product added successfully	Pass
TC-03	Update Product	Data updated	Pass
TC-04	Delete Product	Product removed	Pass
TC-05	View Records	Data displayed correctly	Pass

Products							
Manage your product inventory							
+ Add Product Update Delete Search Low Stock Refresh							
Product ID	Name	SKU	Price	Stock	Reorder	Category	Supplier
CA4390	Camera	MOB-CAM-9569	3000.0	10	5	Mobile Accessories	Xyz productions
CH9502	Charger	MOB-CHA-4022	300.0	5	5	Mobile Accessones	Xyz productions
GL3367	Glass guard	MOB-GLA-2765	150.0	9	5	Mobile Accessories	Xyz productions

8.8 Test Report Summary:

All major functionalities of the InvenTrack system were tested successfully. The system performs efficiently under normal conditions without crashes or data loss.

- System response time is fast
- Data operations are accurate
- GUI is stable and responsive

Overall, the system meets all functional requirements and is ready for deployment.

CHAPTER -9

SYSTEM IMPLEMENTATION

9.1 Objectives of Implementation

System implementation is the process of converting the developed InvenTrack system into a fully operational application. It ensures that the designed system works correctly in a real-world environment.

The main objectives of implementation are:

1. To deploy the inventory management system on user machines
2. To ensure smooth interaction between GUI, backend logic, and database
3. To provide a user-friendly environment for managing inventory operations
4. To verify that all modules function together without errors

9.2 Implementation Plan

The implementation of the system was carried out in a structured manner. Each component was developed, tested, and integrated step by step.

Phase	Activity	Deliverable
Phase 1	Environment Setup	Python and required libraries installed
Phase 2	Database Setup	Tables created using setup.py
Phase 3	GUI Development	Screens designed using Tkinter
Phase 4	Backend Integration	UI connected with database operations
Phase 5	Testing & Validation	System verified for correct functionality

The development followed a modular approach, where each part was completed before moving to the next stage.

9.3 Installation Procedure

Step 1 – Install Python:

Download and install Python from <https://www.python.org/downloads/> (ensure “Add to PATH” selected).

The image shows the Python.org website and the Python 3.14.0 (64-bit) Setup window. The website's 'Downloads' menu is open, highlighting 'Download for Windows' and 'Python 3.14.0'. Below the menu, a table lists active Python releases.

Python version	Maintenance status	First released	End of support	Release schedule
3.14.0	pre-release	2026-10-07 (planned)	2031-10	PEP 790

The setup window shows the 'Install Python 3.14.0 (64-bit)' options. The 'Add python.exe to PATH' checkbox is selected and circled in red.

Python 3.14.0 (64-bit) Setup

Install Python 3.14.0 (64-bit)

Select Install Now to install Python with default settings, or choose Customize to enable or disable features.

NOTE: This installer is being retired and will no longer be available after Python 3.15. [More info](#)

→ **Install Now**
C:\Users\bhanw\AppData\Local\Programs\Python\Python314
Includes IDLE, pip and documentation
Creates shortcuts and file associations

→ **Customize installation**
Choose location and features

☐ Use admin privileges when installing py.exe
☒ **Add python.exe to PATH**

Cancel

Step 2 – Install required libraries:

```
C:\Users\bhanw>pip install bcrypt mysql_connector numpy pandas
Requirement already satisfied: bcrypt in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (5.0.0)
Requirement already satisfied: mysql_connector in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (2.2.9)
Requirement already satisfied: numpy in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (2.4.4)
Requirement already satisfied: pandas in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (2.3.3)
Requirement already satisfied: python-dateutil>=2.8.2 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from pandas) (2025.2)
Requirement already satisfied: six>=1.5 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

Step 3 – Setup Database:

Run the setup file to create database tables:

```
[1/5] Checking required packages...
✓ mysql-connector-python
✓ python-dotenv
✓ bcrypt
All packages ready.

=====
          InvenTrack – First-Time Setup
=====

[2/5] Database Configuration
Enter your MySQL connection details.

DB Host [localhost]: localhost
DB User [root]: root
DB Password:
DB Name [inventrack]: inventrack

.env file written → c:\Users\bhanw\Desktop\InvenTrack\.env

[3/5] Testing database connection...
✓ Connected. Database 'inventrack' is ready.

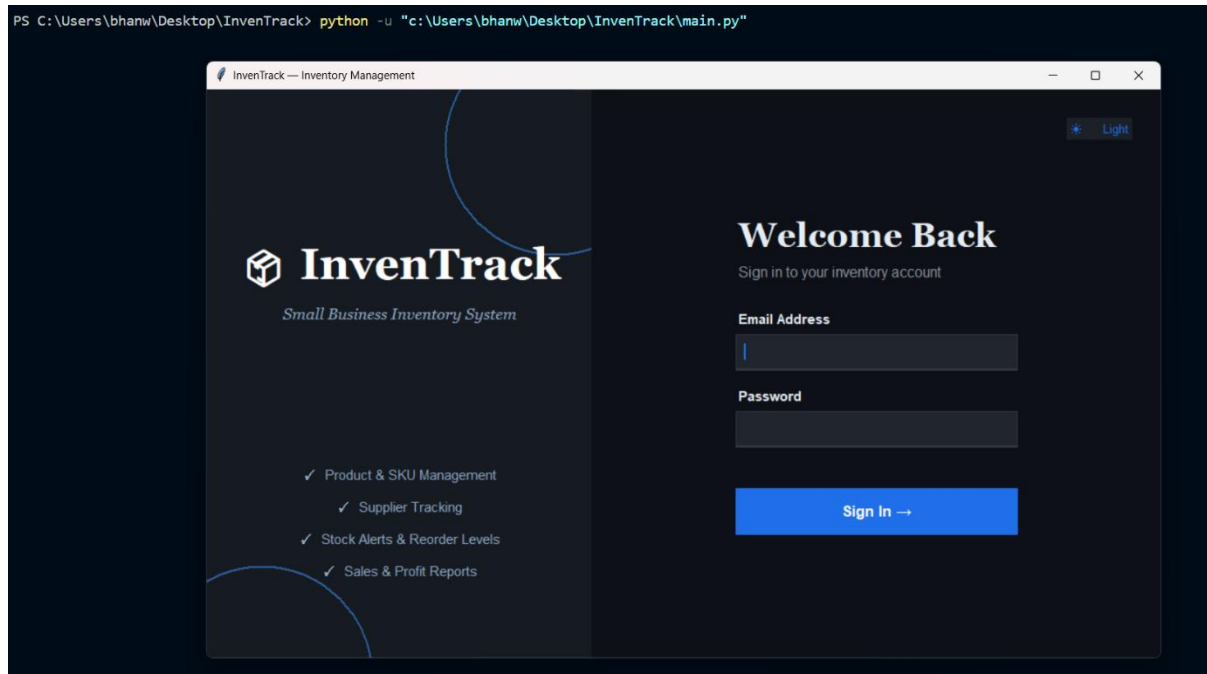
[4/5] Creating database tables...
✓ users
✓ categories
✓ suppliers
✓ product
✓ orders
✓ payment

[5/5] Default Admin Account
i An admin account already exists – skipping creation.
```

Step 4 – Run Application:

Start the system using python main.py

The graphical user interface will launch automatically.



9.4 Deployment Environment

The system was tested and deployed under the following environment:

- Operating System: Windows 10 / Linux
- Programming Language: Python 3.8+
- IDE: Visual Studio Code / PyCharm
- Database: MySQL (via setup.py)
- Hardware: Minimum 4GB RAM, standard processor

The system runs efficiently on standard machines without requiring high-end hardware. System tested under both standalone and LAN environments.

9.5 Key Implementation Modules

(a) Initialization Module

This module initializes the application and establishes database connections. It ensures that required tables exist before execution.

```
TABLES = {
    "users": ""
        CREATE TABLE IF NOT EXISTS users (
            user_id    VARCHAR(20)  PRIMARY KEY,
            name       VARCHAR(50)  NOT NULL,
            email      VARCHAR(100) UNIQUE NOT NULL,
            password   VARCHAR(255) NOT NULL,
            role       ENUM('admin','user') NOT NULL DEFAULT 'user',
            is_deleted TINYINT(1) DEFAULT 0,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
    "",
    "categories": ""
        CREATE TABLE IF NOT EXISTS categories (
            category_id INT AUTO_INCREMENT PRIMARY KEY,
            category_name VARCHAR(100) UNIQUE NOT NULL,
            description  VARCHAR(255)
        )
    "",
    "suppliers": ""
        CREATE TABLE IF NOT EXISTS suppliers (
            supplier_id INT AUTO_INCREMENT PRIMARY KEY,
            supplier_name VARCHAR(100) NOT NULL,
            contact_name VARCHAR(100),
            phone        VARCHAR(20),
            email        VARCHAR(100),
            address      VARCHAR(255),
            is_deleted   TINYINT(1) DEFAULT 0,
            created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
    "",
    "product": ""
        CREATE TABLE IF NOT EXISTS product (
            product_id VARCHAR(20)  PRIMARY KEY,
            product_name VARCHAR(100) NOT NULL,
            sku         VARCHAR(50)  UNIQUE,
            price       FLOAT        NOT NULL,
            cost_price  FLOAT        DEFAULT 0,

```

```

        stock          INT          NOT NULL DEFAULT 0,
        reorder_level INT          DEFAULT 5,
        category_id    INT,
        supplier_id     INT,
        is_deleted      TINYINT(1) DEFAULT 0,
        created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (category_id) REFERENCES categories(category_id),
        FOREIGN KEY (supplier_id) REFERENCES suppliers(supplier_id)
    )
    """
    "orders": """
    CREATE TABLE IF NOT EXISTS orders (
        order_id      INT AUTO_INCREMENT PRIMARY KEY,
        user_id        VARCHAR(20),
        product_id     VARCHAR(20),
        quantity       INT NOT NULL,
        order_status   ENUM('placed','shipped','delivered','cancelled','refunded') NOT NULL DEFAULT 'placed',
        created_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (user_id) REFERENCES users(user_id),
        FOREIGN KEY (product_id) REFERENCES product(product_id)
    )
    """
    "payment": """
    CREATE TABLE IF NOT EXISTS payment (
        payment_id     INT AUTO_INCREMENT PRIMARY KEY,
        order_id        INT UNIQUE,
        amount          FLOAT NOT NULL,
        payment_status  ENUM('pending','completed','failed','refunded') DEFAULT 'pending',
        paid_at         TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (order_id) REFERENCES orders(order_id)
    )
    """
}

Windsurf: Refactor | Explain | Generate Docstring | X
def create_tables(host, user, password, db_name):
    print("\n[4/5] Creating database tables...")
    try:

```

```

def create_tables(host, user, password, db_name):
    print("\n[4/5] Creating database tables...")
    try:
        conn = get_connection(host, user, password, db_name)
        cur = conn.cursor()
        for table_name, ddl in TABLES.items():
            cur.execute(ddl)
            conn.commit()
            print(f"    ✓ {table_name}")
        cur.close()
        conn.close()
    except mysql.connector.Error as e:
        print(f"    X Error creating tables: {e}")
        sys.exit(1)

```

(b) GUI Module

Implemented using Tkinter, this module handles all user interactions including navigation between screens such as dashboard, product management, and reports.

```
class InvenTrackApp(tk.Tk):
    Windsurf: Refactor | Explain | Generate Docstring | ✕
    def __init__(self):
        super().__init__()
        self.title("InvenTrack – Inventory Management")
        self.geometry("1200x720")
        self.minsize(960, 620)
        self.configure(bg=T()["bg"])
        self.resizable(True, True)
        self._show_login()

    Windsurf: Refactor | Explain | Generate Docstring | ✕
    def _clear(self):
        for w in self.winfo_children():
            w.destroy()

    Windsurf: Refactor | Explain | Generate Docstring | ✕
    def _show_login(self):
        self._clear()
        self.configure(bg=T()["bg"])
        LoginPage(self)

    Windsurf: Refactor | Explain | Generate Docstring | ✕
    def _show_main(self, role):
        self._clear()
        self.configure(bg=T()["bg"])
        MainApp(self, role)
```

c) Inventory Management Module

Handles operations such as adding, updating, deleting, and viewing inventory items. It interacts directly with the database.

```
class ProductsPage(tk.Frame):
    Windsurf: Refactor | Explain | Generate Docstring | X
    def __init__(self, master):
        super().__init__(master, bg=T()["bg"], padx=28, pady=20)
        self.pack(fill="both", expand=True)
        self._build()

    Windsurf: Refactor | Explain | Generate Docstring | X
    def _build(self):
        page_header(self, "Products", "Manage your product inventory")
        bf = tk.Frame(self, bg=T()["bg"])
        bf.pack(fill="x", pady=(0, 10))
        if current_user_role == "admin":
            action_btn(bf, "+ Add Product", self._add).pack(side="left", padx=(0,6))
            action_btn(bf, "🔄 Update", self._update, "#2E5F9E").pack(side="left", padx=(0,6))
            action_btn(bf, "🗑 Delete", self._delete, T()["danger"]).pack(side="left", padx=(0,6))
        action_btn(bf, "🔍 Search", self._search, T()["warning"]).pack(side="left", padx=(0,6))
        action_btn(bf, "⚠ Low Stock", self._show_low, T()["warning"]).pack(side="left", padx=(0,6))
        action_btn(bf, "🔄 Refresh", self._load, T()["success"]).pack(side="right")

        self.tree = styled_table(self, ["Product ID", "Name", "SKU", "Price", "Stock", "Reorder", "Category", "Supplier"])
        self.tree.column("Product ID", width=90)
        self.tree.column("SKU", width=130)
        self.tree.column("Price", width=70)
        self.tree.column("Stock", width=60)
        self.tree.column("Reorder", width=70)
        self._load()

    Windsurf: Refactor | Explain | Generate Docstring | X
    def _load(self, filter_name=None, low_only=False):
        for r in self.tree.get_children():
            self.tree.delete(r)
        try:
            where = "WHERE p.is_deleted=0"
            vals = []
            if filter_name:
                where += " AND p.product_name LIKE %s"
                vals.append(f"%{filter_name}%")
            if low_only:
                where += " AND p.stock <= p.reorder_level"
            cursor.execute(f"""
                SELECT p.product_id, p.product_name, p.sku, p.price, p.stock,
                       p.reorder_level, c.category_name, s.supplier_name
                FROM product p
                LEFT JOIN categories c ON p.category_id=c.category_id
                LEFT JOIN suppliers s ON p.supplier_id=s.supplier_id
                {where} ORDER BY p.product_name
            """, vals)
            for r in cursor.fetchall():
                stock, reorder = r[4], r[5]
                tag = "out" if stock == 0 else ("low" if stock <= reorder else "")
                self.tree.insert("", "end", values=(r[0], r[1], r[2] or "", r[3], r[4], r[5], r[6] or "", r[7] or ""), tags=(tag,))
            self.tree.tag_configure("out", foreground=T()["out_stock"])
            self.tree.tag_configure("low", foreground=T()["low_stock"])
        except mysql.connector.Error as e:
            messagebox.showerror("Error", str(e))
```

(d) Database Module

Executes SQL queries for storing and retrieving data. Ensures data consistency and integrity.

```
def submit():
    n, p_s, c_s, s_s, r_s = name_v.get().strip(), price_v.get().strip(), cost_v.get().strip(), stock_v.get().strip(), reorder_v.get().strip()
    if not all([n, p_s, s_s]):
        messagebox.showwarning("Warning", "Name, Price and Stock are required.", parent=win)
        return
    try:
        price = float(p_s); cost = float(c_s) if c_s else 0
        stock = int(s_s); reorder = int(r_s) if r_s else 5
        cat_id = cat_map.get(cat_v.get())
        supp_id = supp_map.get(supp_v.get())
        pid = generate_product_id(n)
        sku = generate_sku(n, cat_id)
        cursor.execute("""INSERT INTO product (product_id, product_name, sku, price, cost_price, stock, reorder_level, category_id, supplier_id)
        VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s)""",
        (pid, n, sku, price, cost, stock, reorder, cat_id, supp_id))
        connect.commit()
        messagebox.showinfo("Added", f"Product added!\nID: {pid}\nSKU: {sku}", parent=win)
        win.destroy(); self.load()
    except ValueError:
        messagebox.showerror("Error", "Invalid number format.", parent=win)
    except mysql.connector.Error as e:
        messagebox.showerror("Error", str(e), parent=win)
```

(e) Event Handling Module

Manages user actions such as button clicks and triggers corresponding backend functions.

```
def _add(self):
    win = FormWindow(self, "Add Product")
    parent = win.scroll_frame # ☒ SINGLE PARENT SOURCE

    # TEST: Verify parent is working
    tk.Label(parent, text="✓ FORM ACTIVE", bg="yellow", fg="black", font=("Helvetica", 9, "bold")).pack(pady=5)

    name_v = tk.StringVar(); price_v = tk.StringVar(); cost_v = tk.StringVar()
    stock_v = tk.StringVar(); reorder_v = tk.StringVar(value="5")
    form_entry(parent, "Product Name", name_v)
    form_entry(parent, "Selling Price (₹)", price_v)
    form_entry(parent, "Cost Price (₹)", cost_v)
    form_entry(parent, "Stock Quantity", stock_v)
    form_entry(parent, "Reorder Alert Level", reorder_v)

    cats = self._get_categories()
    cat_map = {c[1]: c[0] for c in cats}
    cat_v = tk.StringVar()
    form_dropdown(parent, "Category", cat_v, [c[1] for c in cats] + ["- None -"])

    supps = self._get_suppliers()
    supp_map = {s[1]: s[0] for s in supps}
    supp_v = tk.StringVar()
    form_dropdown(parent, "Supplier", supp_v, [s[1] for s in supps] + ["- None -"])
```

9.6 Challenges During Implementation

Challenge	Cause	Solution
Database connection errors	Incorrect credentials	Fixed connection configuration
GUI lag	Blocking operations	Optimized function calls
Data inconsistency	Improper validation	Added input validation checks
Duplicate entries	Missing constraints	Implemented validation logic

Welcome Back

Sign in to your inventory account

Email Address

admin@gmail.com

Password

•••••

Invalid credentials. 2 attempt(s) left.

Sign In →

9.7 Tools and Technologies Used

- Programming Language: Python
- GUI Framework: Tkinter
- Database: MySQL
- IDE: VS Code / PyCharm
- Libraries: mysql-connector-python

These tools ensured efficient development and smooth system performance.

9.8 User Training and Support

The system is designed to be user-friendly, requiring minimal training. Users can easily perform tasks such as:

- Adding new products
- Updating inventory
- Viewing records
- Managing data

A short demonstration is sufficient for users to operate the system effectively.

9.9 Maintenance and Backup

- Regular database backups are recommended
- System logs help track errors
- Software updates can be applied easily

Proper maintenance ensures long-term system reliability.

9.10 Implementation Outcome

The InvenTrack system was successfully implemented and tested. All modules function correctly and meet the specified requirements.

- System is stable and reliable
- Data operations are accurate
- GUI is responsive and easy to use

The project demonstrates an efficient solution for inventory management using Python and database integration.

CHAPTER -10

CONCLUSION

10.1 CONCLUSION

The InvenTrack system has been successfully designed and implemented as a comprehensive inventory management solution. The project achieves its primary objective of automating inventory-related operations such as adding, updating, deleting, and viewing product records through a user-friendly graphical interface.

The system integrates frontend and backend components effectively, ensuring smooth interaction between the user interface, application logic, and database. By using Python and MySQL, the application provides a reliable and efficient platform for managing inventory data in a structured manner.

One of the major strengths of the system is its simplicity and ease of use. The interface is designed in such a way that even users with minimal technical knowledge can operate it without difficulty. Features such as real-time data display, validation mechanisms, and structured record management improve the overall usability of the system.

The implementation of proper data handling techniques ensures accuracy and consistency in records. The system also minimizes manual effort and reduces the chances of human error, making it more efficient than traditional inventory management methods.

From an academic perspective, this project provides practical exposure to important concepts such as GUI development, database connectivity, modular programming, and software testing. It also demonstrates how theoretical knowledge can be applied to develop real-world applications.

In conclusion, the InvenTrack system serves as an effective, reliable, and scalable solution for inventory management, fulfilling all the objectives defined at the beginning of the project.

10.2 Future Scope

Although the current system fulfills all essential requirements, there are several opportunities for further enhancement and expansion.

- Integration with cloud-based databases to enable remote access and real-time synchronization
- Development of a web-based or mobile application for better accessibility
- Implementation of barcode or QR code scanning for faster inventory updates
- Addition of advanced analytics and graphical reports for better decision-making
- Multi-user access with role-based permissions for large organizations
- Integration with billing and accounting systems

These enhancements would improve scalability, usability, and functionality, making the system suitable for larger and more complex environments.

CHAPTER -11

BIBLIOGRAPHY AND REFERENCES

11.1 Bibliography

- A. Guido van Rossum (1991). Python Programming Language. Python Software Foundation.
- B. MySQL Documentation – <https://dev.mysql.com/doc/>
- C. Tkinter User Guide – <https://docs.python.org/3/library/tkinter.html>
- D. Bcrypt Documentation – <https://pypi.org/project/bcrypt/>
- E. Python Database Connectivity – <https://docs.python.org/3/library/sqlite3.html>

11.2 Software and Hardware References

- Python Version: 3.8+
- MySQL Database Server
- Tkinter (Built-in GUI Library)
- mysql-connector-python
- Hardware: Intel Core i3 / Ryzen 3, 4 GB RAM

11.3 Sample Attendance Log

Item ID	Item Name	Quantity	Category	Price
ITM001	Keyboard	25	Accessories	500
ITM002	Mouse	40	Accessories	300

11.4 Acknowledged Resources

- Open-source contributors of Python and MySQL
- Documentation and community support from Python developers
- Academic guidance from faculty members of Computer Applications Department
- Online programming forums and tutorials for troubleshooting

11.5 Final Remarks

This report represents a complete technical documentation of the InvenTrack Inventory Management System.

It demonstrates the practical implementation of GUI-based application development, database integration, and software engineering principles. The system can serve both as an academic project and as a foundation for developing advanced inventory management solutions in real-world environments.